

Санкт-Петербургский государственный университет

Направление "Большие данные и распределенная цифровая платформа"

Лабораторная работа по дисциплине

Алгоритмы и структуры данных

**"Решение задачи о коммивояжере с помощью метода
ближайшего соседа."**

Выполнил:

Зайнуллин Мансур Альбертович

Группа: 23.Б16-пу

Руководитель:

Дик Александр Геннадьевич

ассистент кафедры компьютерного

моделирования

и многопоточных систем

Санкт-Петербург

2025

Оглавление

1	Цель работы	3
2	Теоретические основы	4
3	Алгоритмическая реализация	5
3.1	Псевдокод базового алгоритма	5
3.2	Обработка особых случаев	5
3.3	Вычислительная сложность	6
3.4	Модифицированная версия алгоритма	6
3.5	Блок-схема	6
4	Описание программы	9
4.1	Структура проекта	9
4.2	Спецификация программы	9
4.2.1	graph.py	9
4.2.2	Спецификация модуля graph.py	9
4.2.3	main.py	12
4.2.4	nearest_neighbor_method.py	13
4.2.5	colorGUI.py	14
5	Контрольный пример	15
6	Тестирование алгоритма ближайшего соседа и его модификации	19
6.1	Методология тестирования	19
6.2	Результаты тестирования	20
6.2.1	Граф с 4 вершинами и 8 рёбрами	20
6.2.2	Граф с 20 вершинами и 186 рёбрами	21
6.2.3	Граф с 100 вершинами и 3395 рёбрами	24
6.2.4	Граф с 500 вершинами и 100000 рёбрами	26
6.3	Анализ результатов	28
7	Вывод	29
8	Листинг программы	30

8.1	main.py	30
8.2	graph.py	31
8.3	nearest_neighbor_method.py	33
8.4	colorGUI.py	35
9	Полезные ссылки	49

1 Цель работы

Целью данной работы является разработка и анализ алгоритма ближайшего соседа для решения задачи коммивояжёра в ориентированных взвешенных графах. А также реализация модификации ”сверхбыстрый отжиг” и сравнение его с классическим алгоритмом имитации отжига.

2 Теоретические основы

Алгоритм ближайшего соседа — метод решает задачу коммивояжёра в графах.

Основные этапы:

Работа алгоритма состоит из следующих этапов:

1. Инициализация

- Выбирается начальная вершина графа (случайно или заданная пользователем).
- Создается пустой список пути, куда добавляется начальная вершина.
- Все вершины графа помечаются как непосещенные, кроме начальной.

2. Основной цикл

- Для текущей вершины анализируются все исходящие ребра.
- Из непосещенных соседних вершин выбирается та, ребро к которой имеет минимальный вес.
- Выбранная вершина добавляется в путь и помечается как посещенная.
- Текущая вершина обновляется до выбранной вершины.
- Шаг повторяется, пока не будут посещены все вершины графа.

3. Завершение

- После посещения всех вершин добавляется ребро, возвращающее путь в начальную вершину, если такое ребро существует.
- Формируется замкнутый цикл, и вычисляется суммарный вес пути.
- Если возврат невозможен, алгоритм фиксирует ошибку.

3 Алгоритмическая реализация

3.1 Псевдокод базового алгоритма

```
NearestNeighbor(graph, start_vertex):
    path = [start_vertex]      //
    visited = {start_vertex}   //
    current = start_vertex     //

    visited :
        next_vertex = null
        min_weight =
            (current, neighbor) current:
                neighbor visited < min_weight:
                    min_weight =
                    next_vertex = neighbor
        next_vertex = null:
            " "
        next_vertex path
        next_vertex visited
        current = next_vertex

    (current, start_vertex):
        start_vertex path
        path
    :
```

3.2 Обработка особых случаев

Алгоритм учитывает следующие ситуации:

- **Отсутствие непосещенных соседей:** Если на каком-либо шаге у текущей вершины нет доступных непосещенных соседей, алгоритм завершается с ошибкой, указывая на отсутствие гамильтонова цикла.
- **Невозможность возврата:** Если после посещения всех вершин нет

ребра в начальную вершину, цикл не замыкается, и результат считается некорректным.

- **Полные графы:** Алгоритм работает надежно в случае полных графов, где каждая вершина связана со всеми остальными.

3.3 Вычислительная сложность

- **Временная сложность:** $O(n^2)$, где n — число вершин. На каждом шаге алгоритм просматривает до n ребер для выбора минимального, и таких шагов выполняется $n-1$.
- **Пространственная сложность:** $O(n)$ для хранения пути и множества посещенных вершин.

3.4 Модифицированная версия алгоритма

Для повышения точности может применяться модификация, перебирающая все возможные стартовые вершины:

- Алгоритм запускается n раз, каждый раз с новой начальной вершиной.
- Из всех полученных путей выбирается тот, у которого суммарный вес минимален.
- **Сложность:** $O(n^3)$, так как базовый алгоритм $O(n^2)$ выполняется n раз.

3.5 Блок-схема

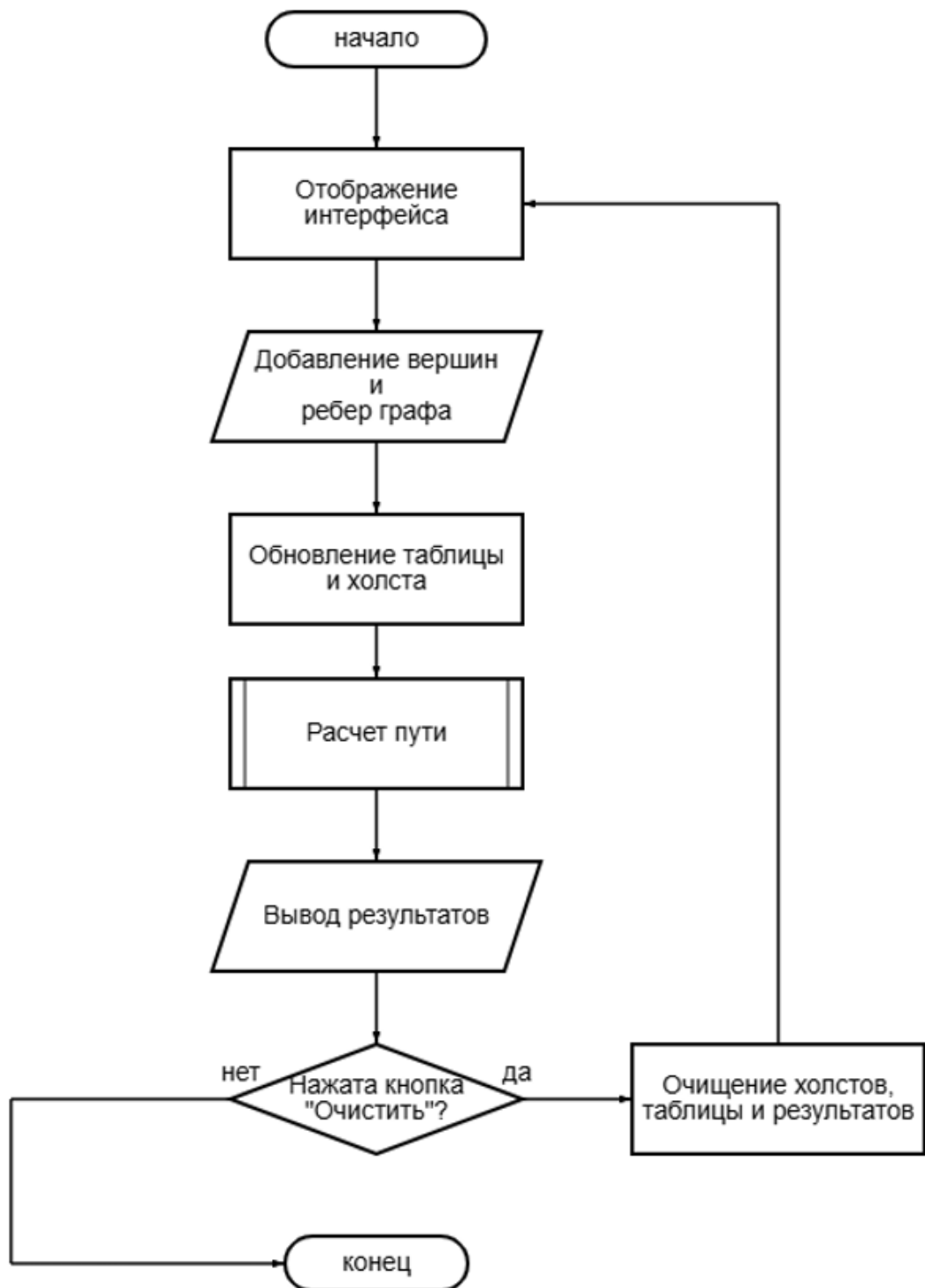


Рис 1. Блок-схема основной программы

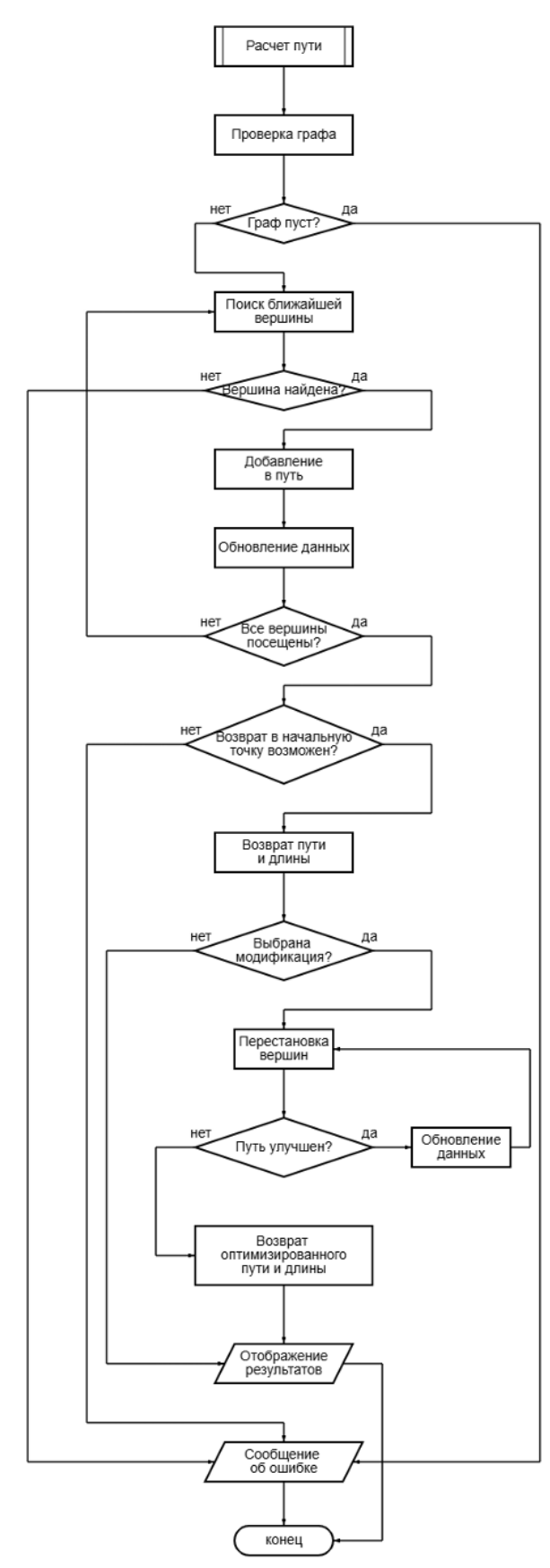


Рис 2. Блок-схема подпрограммы

4 Описание программы

Программа написана на Python 3.11. Графический интерфейс программы написан на Tkinter.

4.1 Структура проекта

1. `graph.py` - ООП реализация графов. Основной класс - `Graph`. Он содержит в себе экземпляры классов `Node` и `Edge`
2. `main.py` - запуск графического интерфейса и создаёт граф с заданным количеством вершин и рёбер если это требуется.
3. `colorGUI.py` - красивый и удобный графический интерфейс для взаимодействия с программой.
4. `nearest_neighbor_method.py` - реализация метода ближайших соседей вместе с его модификацией.

4.2 Спецификация программы

4.2.1 `graph.py`

4.2.2 Спецификация модуля `graph.py`

Table 1: Класс Node

Метод/Атрибут	Описание	Входные/Выходные данные
value	Уникальный идентификатор вершины	— → int
neighbors	Список смежных вершин	— → List[Node]
outgoing_edges	Исходящие рёбра	— → List[Edge]
x, y	Координаты для визуализации	— → float или None
visited	Флаг посещения вершины	— → bool
add_neighbor(node)	Добавляет вершину в список соседей	node (Node) → Обновление списка neighbors
sorted_edges()	Сортирует исходящие рёбра по возрастанию веса	— → List[Edge] (отсортированный)

Table 2: Класс Edge

Метод/Атрибут	Описание	Входные/Выходные данные
from_node	Начальная вершина ребра	— → Node
to_node	Конечная вершина ребра	— → Node
weight	Вес ребра (по умолчанию 1)	— → int

Table 3: Класс Graph

Метод/Атрибут	Описание	Входные/Выходные данные
<code>directed</code>	Флаг ориентированности графа	<code>— → bool</code> (True по умолчанию)
<code>nodes</code>	Список всех вершин графа	<code>— → List[Node]</code>
<code>edges</code>	Список всех рёбер графа	<code>— → List[Edge]</code>
<code>add_node(value)</code>	Создаёт и добавляет новую вершину	<code>value (int) → Созданный объект Node</code>
<code>add_edge(from, to, weight)</code>	Добавляет направленное ребро между вершинами	<code>from (Node), to (Node), weight (int) → Создание Edge + обратного ребра для неориентированных графов</code>
<code>display()</code>	Выводит текстовое представление графа	<code>— → Вывод в консоль списков nodes/edges</code>

4.2.3 main.py

Table 4: Спецификация main.py

Объект/Функция	Описание	Входные/Выходные данные
<code>create_graph()</code>	Генерирует ориентированный граф со случайной топологией	<code>num_nodes</code> (int), <code>num_edges</code> (int), <code>canvas_width</code> (int), <code>canvas_height</code> (int) → Объект <code>Graph</code>
<code>main()</code>	Инициализирует GUI	— → Запуск графического интерфейса

4.2.4 nearest_neighbor_method.py

Table 5: Ключевые функции

Объект/Функция	Описание	Входные/Выходные данные
<code>clear_edge_with_visited_node()</code>	Фильтрация рёбер для исключения посещённых вершин	<code>edges (List[Edge])</code> → <code>List[Edge]</code> — рёбра, ведущие в непосещённые вершины
<code>nearest_neighbor_method()</code>	Реализация алгоритма ближайшего соседа	<code>graph (Graph)</code> , <code>start_node (Node)</code> → Кортеж: Граф с найденным циклом (<code>Graph</code>), Вес пути (<code>int</code>), Статус (<code>str</code>), Путь (<code>str</code>)

Table 6: Возвращаемые статусы алгоритма

Статус	Условие возникновения
”Гамильтонов цикл успешно найден.”	Все вершины пройдены, цикл замкнут
”Ошибка: граф пуст.”	Граф не содержит вершин
”Ошибка: неверная стартовая вершина”	Указанная стартовая вершина отсутствует в графе
”Ошибка: Нет доступных узлов для продолжения.”	Нет непосещённых вершин для продолжения пути
”Ошибка: Нет ребра для замыкания цикла.”	Отсутствует ребро возврата в стартовую вершину

4.2.5 colorGUI.py

Table 7: Основные компоненты интерфейса

Компонент	Назначение	Тип элемента
input_graph_frame	Панель для отображения исходного графа	LabelFrame + Canvas
output_graph_frame	Панель для визуализации результирующего пути	LabelFrame + Canvas
tree	Таблица рёбер с флагами использования в решении	ttk.Treeview
modification_check	Переключатель модификации алгоритма (перебор всех стартовых вершин)	Checkbutton
time_entry	Поле вывода времени выполнения алгоритма (формат: X.XXXX сек)	Entry (readonly)
path_text	Текстовое поле с найденным путём (формат: "0-1-2-0")	Text (disabled)
path_length_entry	Поле с суммарным весом пути	Entry (readonly)

Table 8: Ключевые методы класса LabApp

Метод	Описание	Входные/Выходные данные
<code>create_widgets()</code>	Инициализация всех элементов интерфейса	— → Построение GUI
<code>on_canvas_click(event)</code>	Обработка кликов: создание вершин/рёбер	<code>event</code> (координаты <code>x,y</code>) → Добавление Node/Edge в граф
<code>start_algorithm()</code>	Запуск алгоритма (базового или модифицированного)	— → Обновление выходных данных и визуализации
<code>update_edges_table()</code>	Обновление таблицы рёбер с отметками использованных в решении	— → Синхронизация данных в Treeview
<code>draw_graph()</code>	Отрисовка графа на указанном холсте	<code>canvas</code> , <code>graph</code> → Визуализация вершин и рёбер

5 Контрольный пример

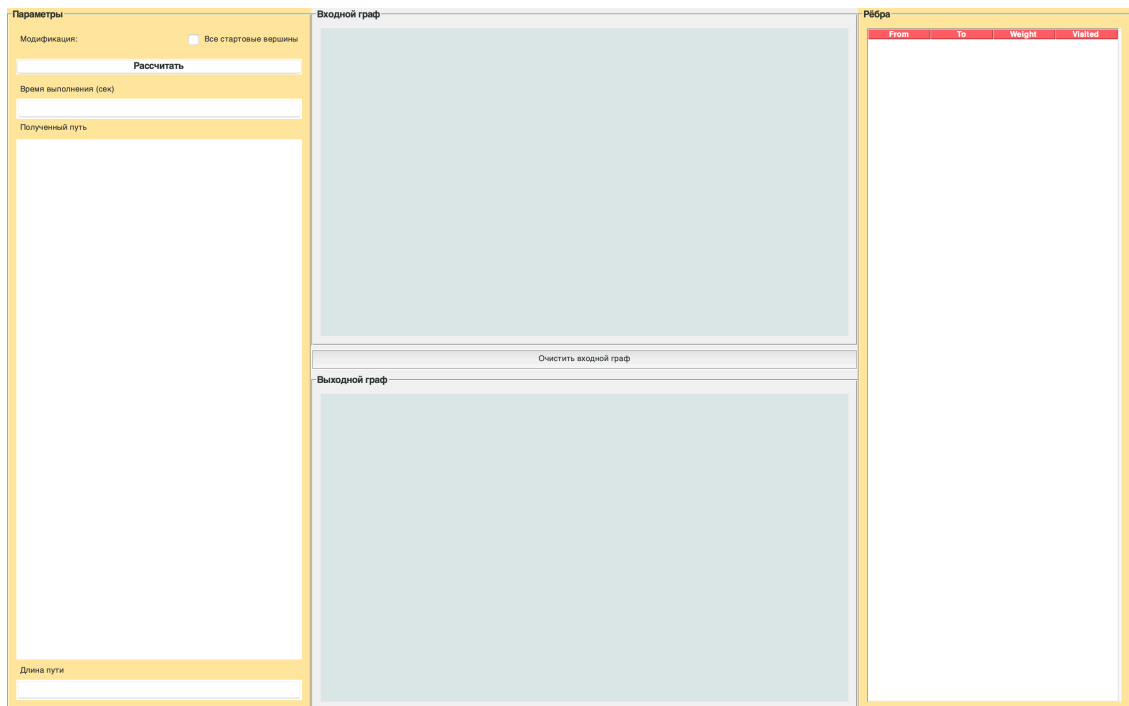


Рис 3. Запуск графического интерфейса через запуск main.py

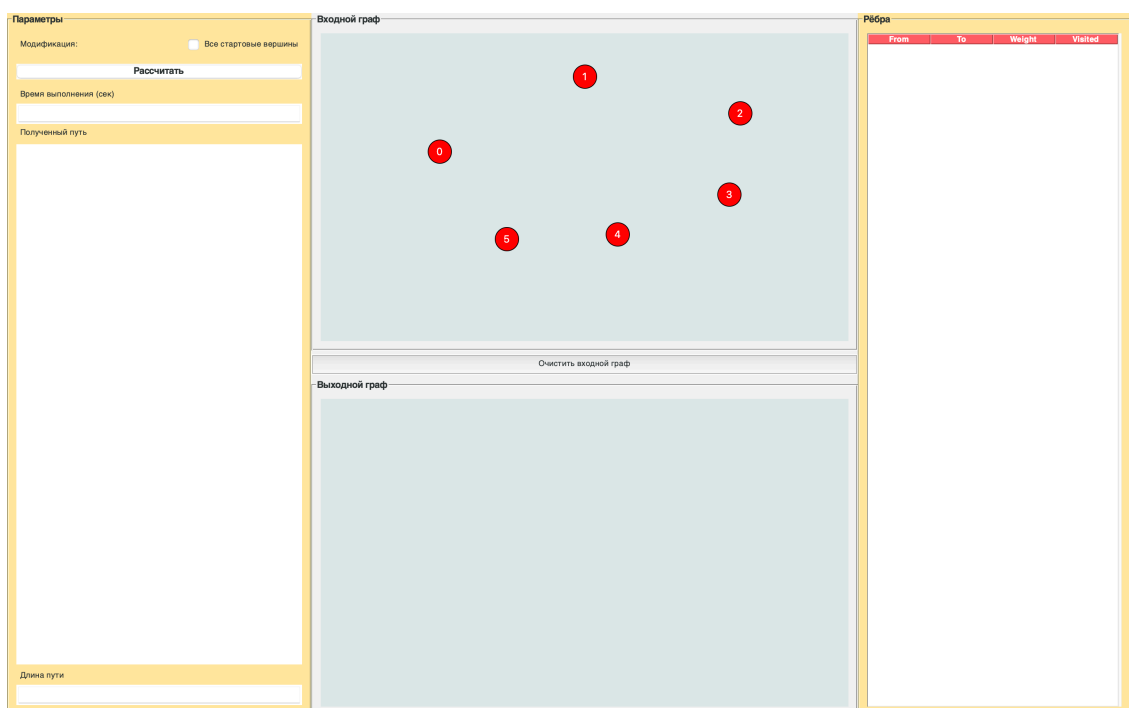


Рис 4. Расставили вершины

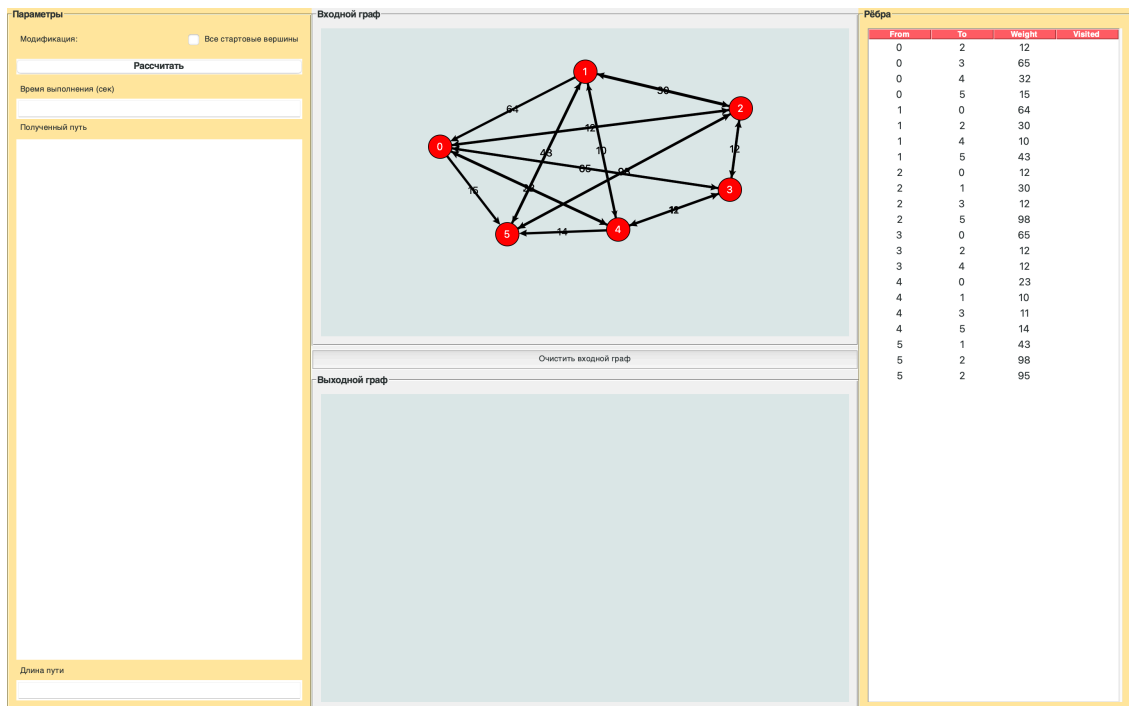


Рис 5. Расставили ориентированные вершины

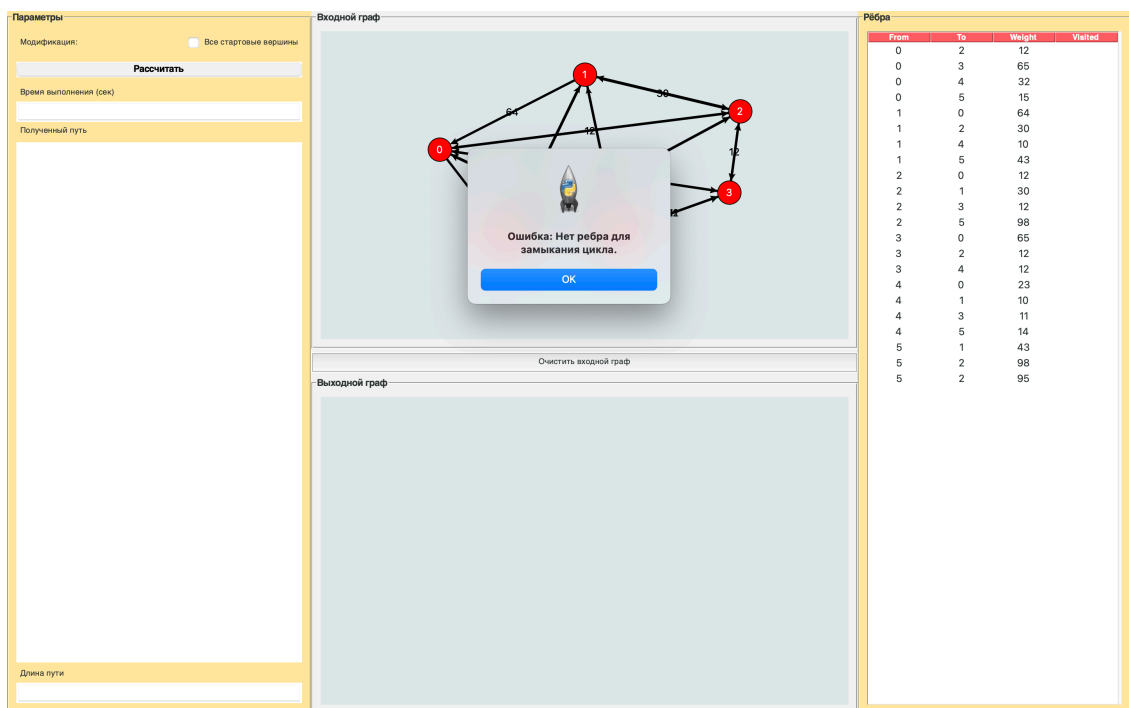


Рис 6. Запустили алгоритм без модификации и он не смог найти для нас гамильтонов цикл

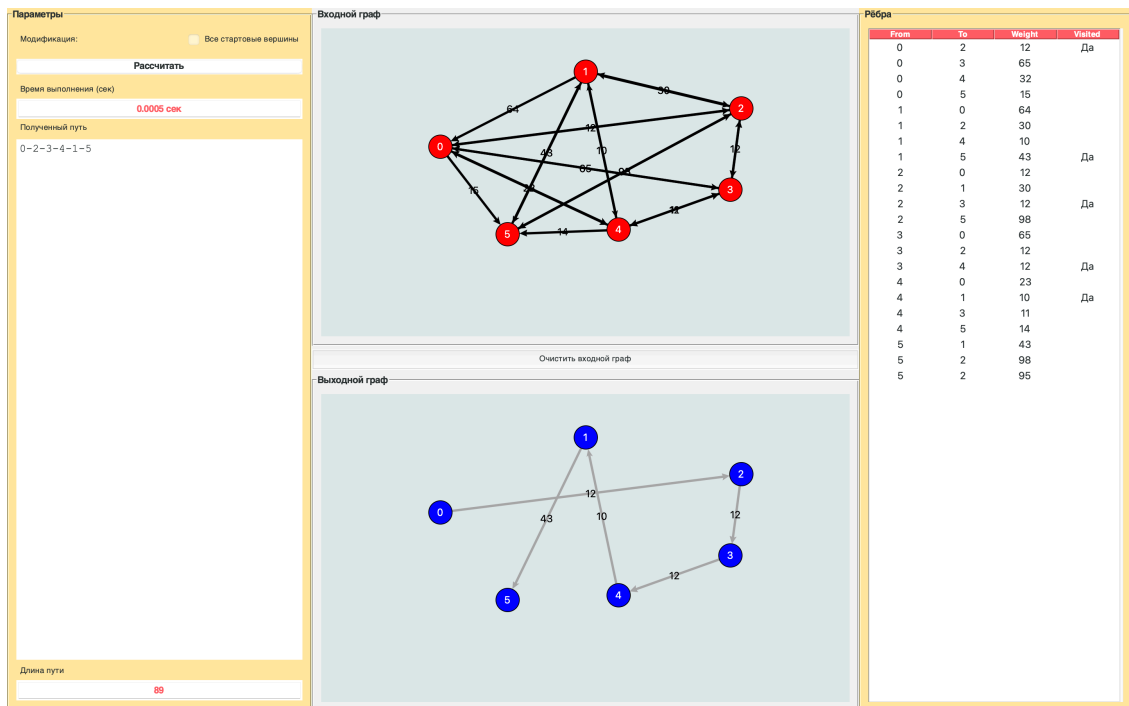


Рис 7. Полученный граф для запуска алгоритма без модификации

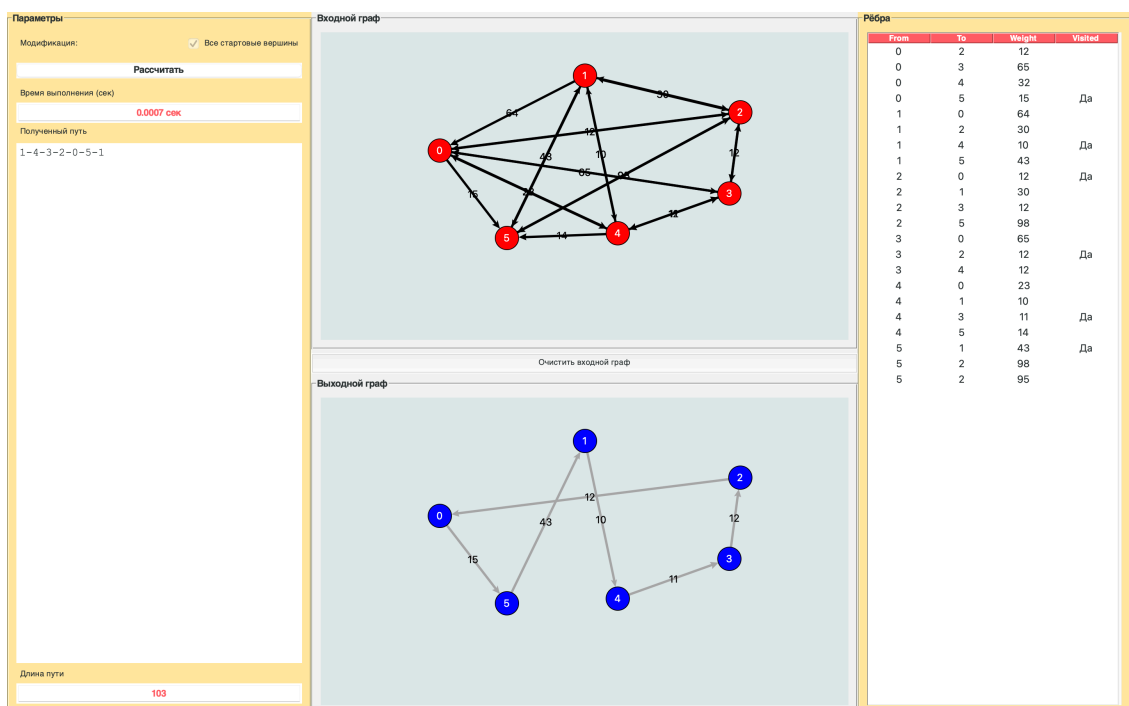


Рис 8. Запустили алгоритм с модификацией и он нашёл для нас гамильтонов цикл с длиной пути 103

6 Тестирование алгоритма ближайшего соседа и его модификации

Для оценки эффективности алгоритма ближайшего соседа и его модификации, перебирающей все стартовые вершины, были проведены испытания на графах различного размера. Тестирование включало измерение длины пути, времени выполнения и успешности нахождения гамильтонова цикла.

6.1 Методология тестирования

- **Графы:** Созданы четыре взвешенных ориентированных графа с 4, 20, 100 и 1000 вершинами.
- **Испытания:** Для каждого графа проведено по 5 запусков базового алгоритма и модификации.

6.2 Результаты тестирования

6.2.1 Граф с 4 вершинами и 8 рёбрами

Базовый алгоритм

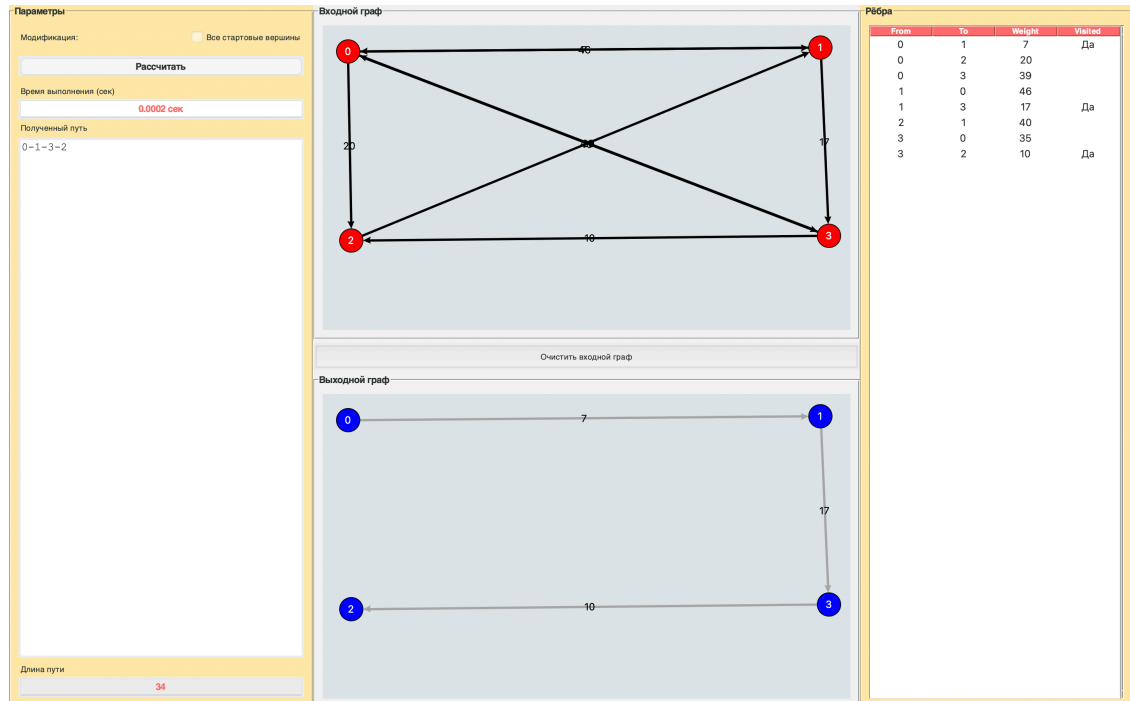


Рис 9. Граф с 4 вершинами и 8 рёбрами

№ испытания	Длина пути	Время выполнения (с)
1	Не найден	0.0002
2	Не найден	0.0001
3	Не найден	0.0001
4	Не найден	0.0001
5	Не найден	0.0003

Table 9: Результаты базового алгоритма (4 вершины)

Найденный путь при первом испытании: 0-1-3-2

Модификация

Найденный путь при первом испытании: 2-1-3-0-2

Вывод: Базовый алгоритм не нашел цикл из-за неудачного выбора стартовой вершины. Модификация стабильно находит цикл минимальной длины.

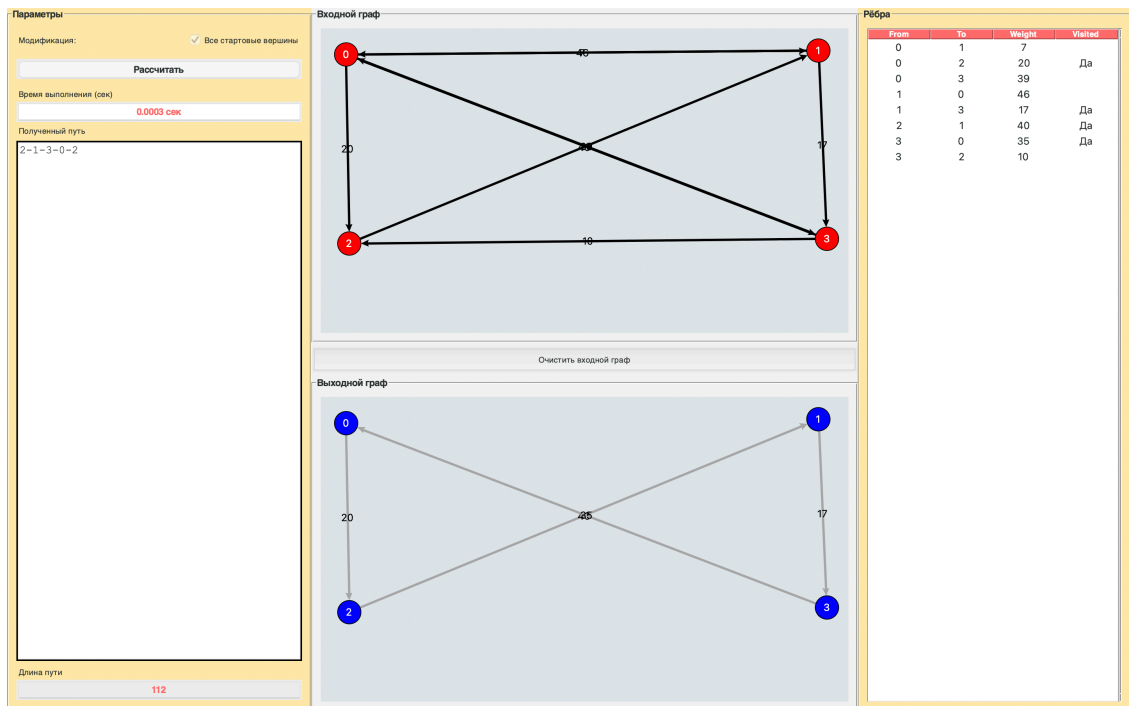


Рис 10. Граф с 4 вершинами и 8 рёбрами (модификация)

№ испытания	Длина пути	Время выполнения (с)
1	112	0.0003
2	112	0.0001
3	112	0.0002
4	112	0.0003
5	112	0.0003

Table 10: Результаты модифицированного алгоритма (4 вершины)

6.2.2 Граф с 20 вершинами и 186 рёбрами

Базовый алгоритм

Найденный путь при первом испытании: 10-12-7-5-18-4-19-0-14-13-3-16-1-17-6-9-2-8-11-15

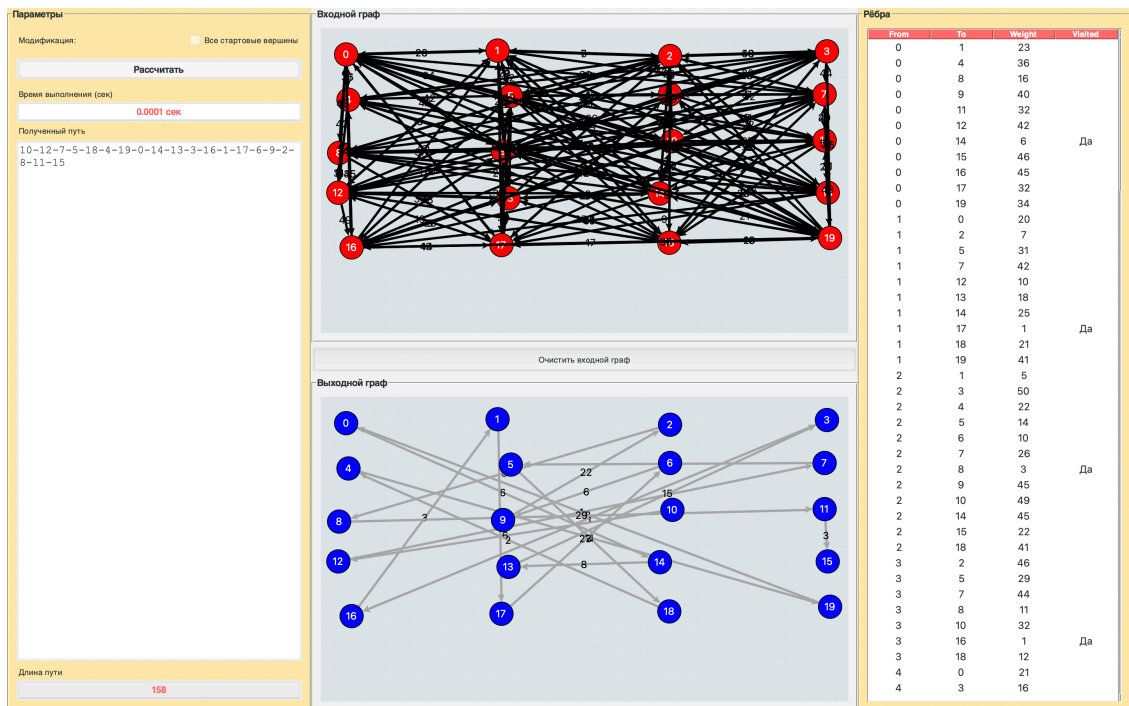


Рис 11. Граф с 20 вершинами и 186 рёбрами

№ испытания	Длина пути	Время выполнения (с)
1	Не найден	0.0001
2	Не найден	0.0001
3	Не найден	0.0002
4	155	0.0002
5	Не найден	0.0001

Table 11: Результаты базового алгоритма (20 вершин)

Модификация

№ испытания	Длина пути	Время выполнения (с)
1	155	0.0017
2	155	0.0012
3	155	0.0020
4	155	0.0013
5	155	0.0032

Table 12: Результаты модифицированного алгоритма (20 вершин)

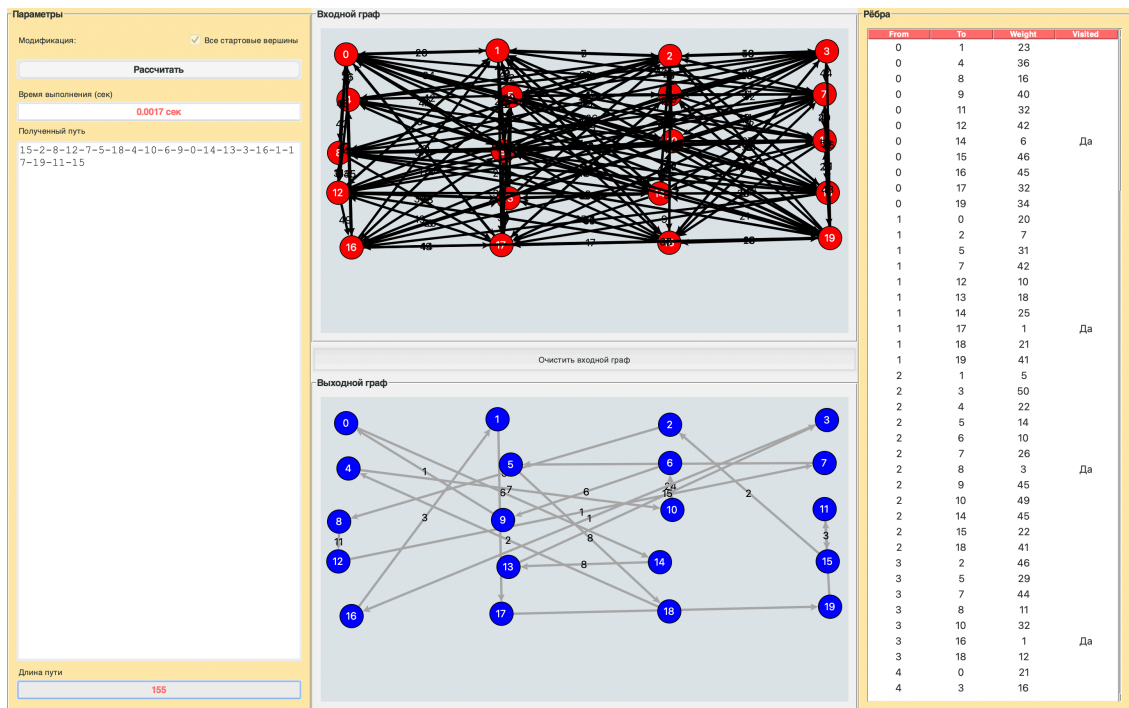


Рис 12. Граф с 20 вершинами и 186 рёбрами (модификация)

Найденный путь при первом испытании: 15-2-8-12-7-5-18-4-10-6-9-0-14-13-3-16-1-17-19-11-15

Вывод: Базовый алгоритм справился лишь в одной из пяти попыток. Модификация нашла цикл.

6.2.3 Граф с 100 вершинами и 3395 рёбрами

Базовый алгоритм

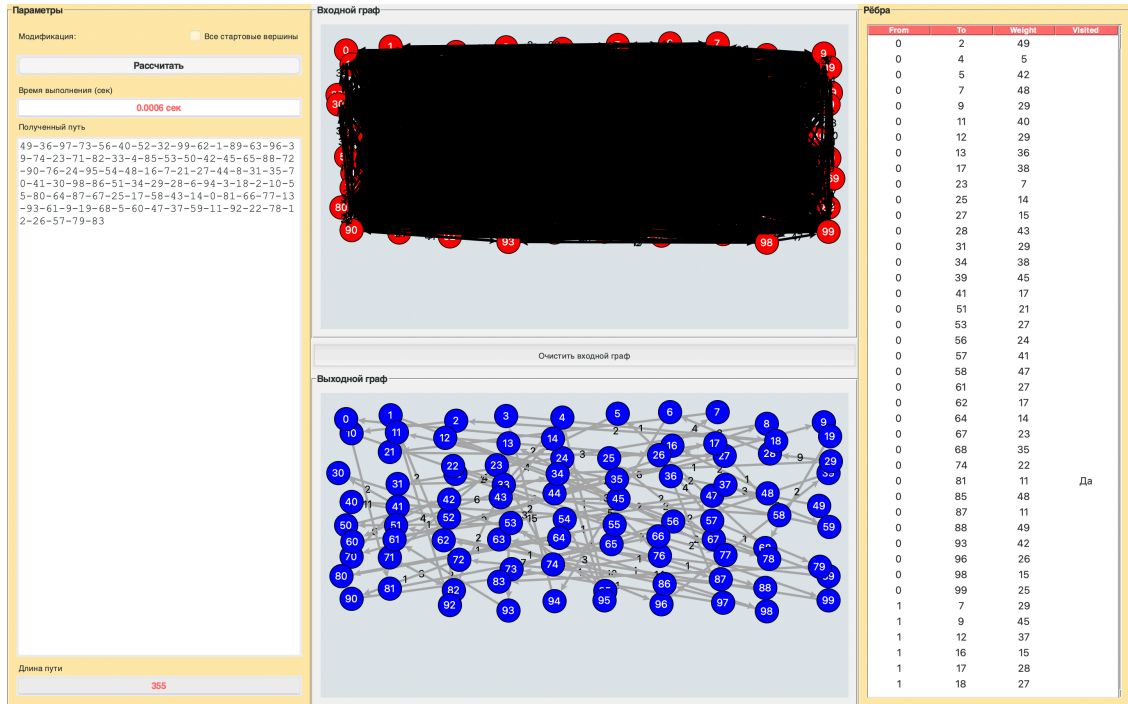


Рис 13. Граф с 100 вершинами и 3395 рёбрами

№ испытания	Длина пути	Время выполнения (с)
1	Не найден	0.0006
2	Не найден	0.0006
3	Не найден	0.0007
4	Не найден	0.0006
5	Не найден	0.0005

Table 13: Результаты базового алгоритма (100 вершин)

Найденный путь при первом испытании: 49-36-97-73-56-40-52-32-99-62-1-89-63-96-39-74-23-71-82-33-4-85-53-50-42-45-65-88-72-90-76-24-95-54-48-16-7-21-27-44-8-31-35-70-41-30-98-86-51-34-29-28-6-94-3-18-2-10-55-80-64-87-67-25-17-58-43-14-0-81-66-77-13-93-61-9-19-68-5-60-47-37-59-11-92-22-78-12-26-57-79-83

Модификация

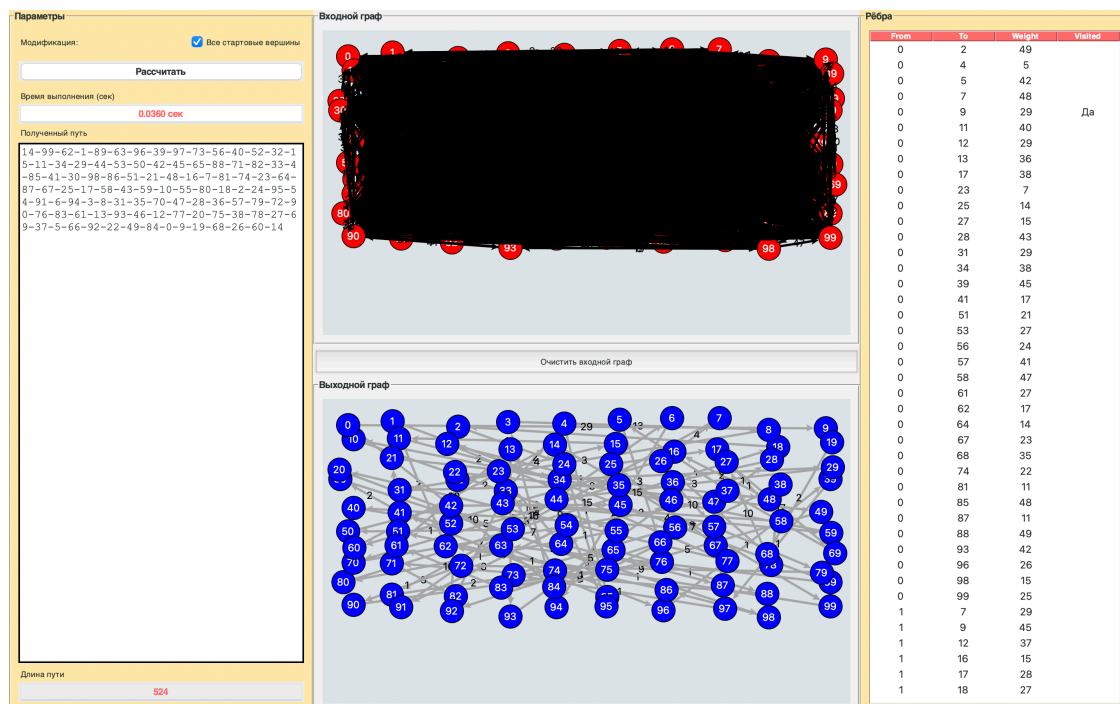


Рис 14. Граф с 100 вершинами и 3395 рёбрами (модификация)

№ испытания	Длина пути	Время выполнения (с)
1	524	0.0360
2	524	0.0300
3	524	0.0399
4	524	0.0356
5	524	0.0334

Table 14: Результаты модифицированного алгоритма (100 вершин)

Найденный путь при первом испытании: 14-99-62-1-89-63-96-39-97-73-56-40-52-32-15-11-34-29-44-53-50-42-45-65-88-71-82-33-4-85-41-30-98-86-51-21-48-16-7-81-74-23-64-87-67-25-17-58-43-59-10-55-80-18-2-24-95-54-91-6-94-3-8-31-35-70-47-28-36-57-79-72-90-76-83-61-13-93-46-12-77-20-75-38-78-27-69-37-5-66-92-22-49-84-0-9-19-68-26-60-14

Вывод: Базовый алгоритм не нашел цикл. Модификация стабильно находит цикл с длиной 524 за 0.03 с.

6.2.4 Граф с 500 вершинами и 100000 рёбрами

Базовый алгоритм

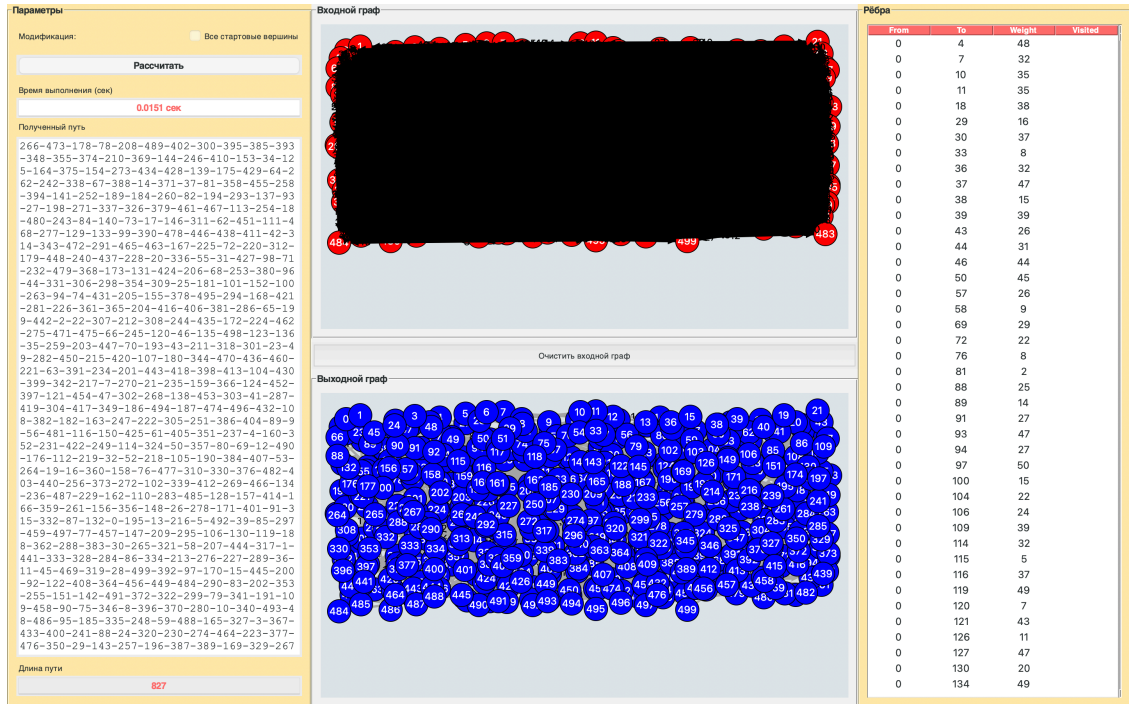


Рис 15. Граф с 500 вершинами и 100000 рёбрами

№ испытания	Длина пути	Время выполнения (с)
1	Не найден	0.0151
2	Не найден	0.0156
3	Не найден	0.0154
4	Не найден	0.0145
5	Не найден	0.0161

Table 15: Результаты базового алгоритма (500 вершин)

Найденный путь при первом испытании: 608-864-855-502-910-258-765-925-817-784-860-285-788-366-546-448-745-44-324-...-471-770-237-898-690-183-130-300-992-364-68-406-821-689-725-125-572-257-766-881-911-614-993-235-786-930-906-559-413-905-528-646-700-8-776-400-433-145-886-417-980-221-840-945-440-241-286-780-835-309-981

Модификация

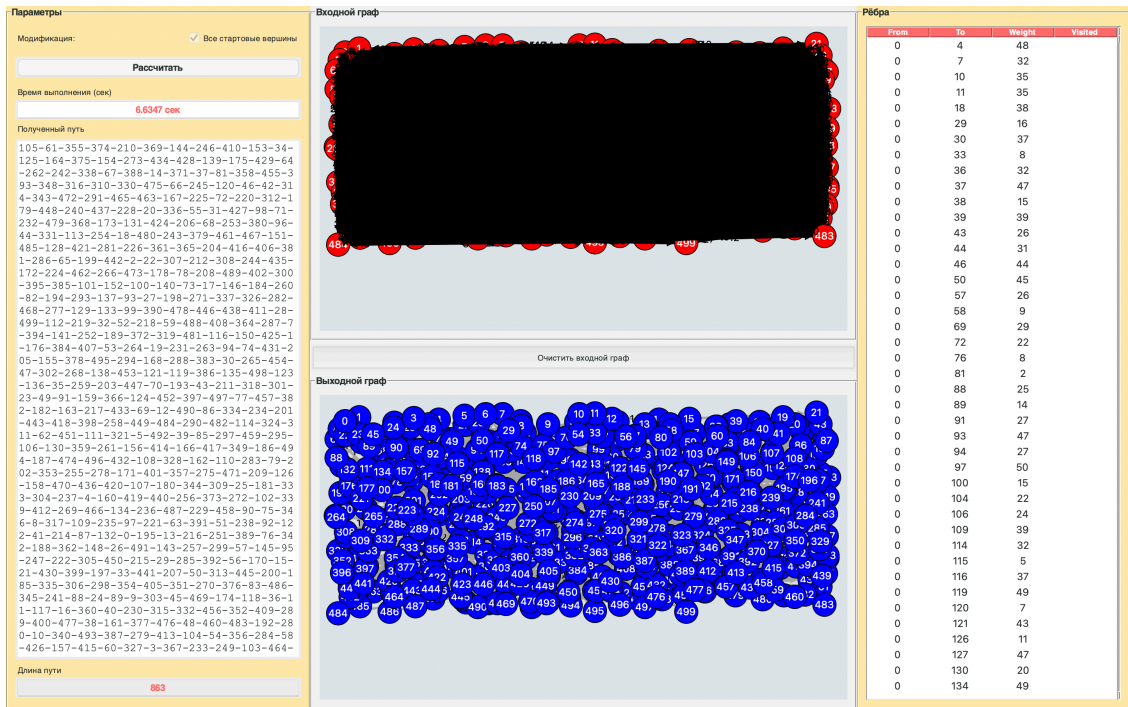


Рис 16. Граф с 500 вершинами и 100000 рёбрами (модификация)

№ испытания	Длина пути	Время выполнения (с)
1	863	6.6347
2	863	6.8464
3	863	6.7080
4	863	6.4343
5	863	6.7434

Table 16: Результаты модифицированного алгоритма (500 вершин)

Найденный путь при первом испытании: 366-546-448-...-626-811-405-401-682-254-637-831-252-495-208-57-822-5-235-786-328-471-732-968-204-900-470-984-286-237-956-110-353-582-183-722-342-364-180-721-152-766-780-30-309-807-424-384-325-528-617-930-957-158-743-440-299-572-257-266-43-395-911-614-70-993-433-512-217-12-449-374-492-800-284-703-419-981-935-835-884-39-729-406-366

6.3 Анализ результатов

- **Эффективность:** Базовый алгоритм не нашел гамильтонов цикл в большинстве случаев из-за зависимости от стартовой вершины, за исключением одного испытания для графа с 20 вершинами. Модификация стабильно находит цикл во всех тестах, если он существует.
- **Время выполнения:** Базовый алгоритм работает быстрее ($O(n^2)$), но часто бесполезен без цикла. Модификация ($O(n^3)$) требует больше времени, особенно на больших графах (до 6.7 с на 500 вершин).
- **Границы применимости:** Базовый алгоритм пригоден только для полных графов с удачным выбором стартовой вершины. Модификация эффективна до 500 вершин; на больших графах время может стать неприемлемым.

Алгоритм ближайших соседей (Базовый)

Граф (вершины, рёбра)	Средняя длина пути	Среднее время выполнения (с)
4, 8	Не найден	0.00016
20, 186	Не найден	0.00014
100, 3395	Не найден	0.0006
500, 100000	Не найден	0.01534

Table 17: Результаты базового алгоритма

Алгоритм ближайших соседей (Модификация)

Граф рёбра)	(вершины,	Средняя длина пути	Среднее время выполнения (с)
4, 8		112	0.00024
20, 186		155	0.00188
100, 3395		524	0.03498
500, 100000		863	6.67336

Table 18: Результаты модифицированного алгоритма

7 Вывод

В данной работе был реализован алгоритм для ближайшего соседа и его модификация, перебирающая все стартовые вершины. Тестирование показало, что базовый алгоритм очень редко справляется тестом, тогда как модификация успешно находила цикл, но она работает медленнее.

8 Листинг программы

8.1 main.py

```
from colorGUI import LabApp
from graph import Graph
import random

def create_graph(num_nodes, num_edges, canvas_width, canvas_height):
    """
    :param num_nodes:
    :param num_edges:
    :param canvas_width:
    :param canvas_height:
    :return:
    """
    graph = Graph(directed=True)

    cols = int(num_nodes**0.5)
    rows = (num_nodes + cols - 1) // cols
    padding = 30

    x_step = (canvas_width - 2*padding) / (cols-1) if cols > 1 else 0
    y_step = (canvas_height - 2*padding) / (rows-1) if rows > 1 else 0

    nodes = []
    for i in range(num_nodes):
        col = i % cols
        row = i // cols
        x = padding + col * x_step + random.randint(-10, 10)
        y = padding + row * y_step + random.randint(-10, 10)
        node = graph.add_node(i)
        node.x = x
        node.y = y
        nodes.append(node)
```

```

all_possible_edges = [(i, j) for i in range(num_nodes)
                       for j in range(num_nodes) if i != j]
selected_edges = random.sample(all_possible_edges, num_edges)

for from_idx, to_idx in selected_edges:
    weight = random.randint(1, 50)
    graph.add_edge(nodes[from_idx], nodes[to_idx], weight)

return graph

def main():
    # graph = create_graph(num_nodes=1000, num_edges=240000,
    #                       ↪ canvas_width=678, canvas_height=395)
    # graph = create_graph(num_nodes=20, num_edges=186, canvas_width=676,
    #                       ↪ canvas_height=300)
    graph = Graph()

    app = LabApp(graph)
    app.attributes('-fullscreen', True)
    app.mainloop()

if __name__ == "__main__":
    main()

```

8.2 graph.py

```

class Node:
    """
    """
    def __init__(self, value):
        self.value = value
        self.neighbors = []
        self.outgoing_edges = []
        self.x = None
        self.y = None
        self.visited = False

    def add_neighbor(self, node):
        """
        """

```

```

        self.neighbors.append(node)

def sorted_edges(self):
    """
    """
    return sorted(self.outgoing_edges, key=lambda edge: edge.weight)

def __repr__(self):
    return f"Node(value={self.value}, (x,y)={({self.x},{self.y}))"

class Edge:
    """
    """
    def __init__(self, from_node, to_node, weight=1):
        self.from_node = from_node
        self.to_node = to_node
        self.weight = weight

    def __repr__(self):
        return f"Edge({self.from_node.value} -> {self.to_node.value},
            ↔ weight={self.weight})"

class Graph:
    """
    """
    def __init__(self, directed=True):
        self.nodes = []
        self.edges = []
        self.directed = directed

    def add_node(self, value):
        """
        """
        node = Node(value)
        self.nodes.append(node)
        return node

    def add_edge(self, from_node, to_node, weight=1):
        """
        """
        forward_edge = Edge(from_node, to_node, weight)
        self.edges.append(forward_edge)
        from_node.add_neighbor(to_node)

```

```

from_node.outgoing_edges.append(forward_edge)
if not self.directed:
    reverse_edge = Edge(to_node, from_node, weight)
    self.edges.append(reverse_edge)
    to_node.add_neighbor(from_node)
    to_node.outgoing_edges.append(reverse_edge)

def display(self):
    """
    """
    print("Nodes:")
    for node in self.nodes:
        print(node)
    print("Edges:")
    for edge in self.edges:
        print(edge)

```

8.3 nearest_neighbor_method.py

```

from graph import Graph, Edge
import random
from typing import List, Tuple

def clear_edge_with_visited_node(edges: List[Edge]) -> List[Edge]:
    """
    , , ."""
    return [edge for edge in edges if not edge.to_node.visited]

def nearest_neighbor_method(graph: Graph, start_node=None) -> Tuple[Graph,
    ↪ int, str, str]:
    """

    :param graph:
    :param start_node:
    :return:
        - ( )
        -
        - (/)

```

```

-                                     "0-4-3-1-2-0"
"""
total_weight = 0
result_graph = Graph(directed=True)

for node in graph.nodes:
    node.visited = False

if not graph.nodes:
    return Graph(), 0, "      :      .", ""

try:
    if start_node is None:
        start_node = random.choice(graph.nodes) if graph.nodes else None
    else:
        if start_node not in graph.nodes:
            return Graph(), 0, "      :      .", ""

    current_node = start_node
    current_node.visited = True
    path = [str(current_node.value)]

    # start_node = random.choice(graph.nodes)
    # current_node = start_node
    # current_node.visited = True
    # path.append(str(current_node.value))

    node_mapping = {}
    start_result_node = result_graph.add_node(start_node.value)
    start_result_node.x, start_result_node.y = start_node.x,
    ↪ start_node.y
    node_mapping[start_node.value] = start_result_node
    previous_result_node = start_result_node

    while len(result_graph.nodes) < len(graph.nodes):
        available_edges =
            ↪ clear_edge_with_visited_node(current_node.sorted_edges())
        if not available_edges:
            message = "      :      ."
            return result_graph, total_weight, message, "-".join(path)

```

```

        next_edge = available_edges[0]
        total_weight += next_edge.weight
        next_node = next_edge.to_node
        next_node.visited = True
        path.append(str(next_node.value))

        if next_node.value not in node_mapping:
            new_node = result_graph.add_node(next_node.value)
            new_node.x, new_node.y = next_node.x, next_node.y
            node_mapping[next_node.value] = new_node
        result_graph.add_edge(previous_result_node,
                               ↪ node_mapping[next_node.value], next_edge.weight)
        previous_result_node = node_mapping[next_node.value]
        current_node = next_node

    closing_edges = [edge for edge in current_node.outgoing_edges if
                     ↪ edge.to_node == start_node]
    if not closing_edges:
        message = "          ."
        return result_graph, total_weight, message, "-".join(path)

    closing_edge = min(closing_edges, key=lambda e: e.weight)
    total_weight += closing_edge.weight
    result_graph.add_edge(previous_result_node,
                           ↪ node_mapping[start_node.value], closing_edge.weight)
    path.append(str(start_node.value))

    message = "          ."
    return result_graph, total_weight, message, "-".join(path)

finally:
    for node in graph.nodes:
        node.visited = False

```

8.4 colorGUI.py

```

import tkinter as tk
from tkinter import simplifiedialog, messagebox, ttk

```



```

import math
import time
import platform #
from nearest_neighbor_method import nearest_neighbor_method
from graph import Graph

class LabApp(tk.Tk):
    def __init__(self, graph):
        super().__init__()
        self.title(" ")
        self.geometry("1250x700")

        self.columnconfigure(0, weight=1, minsize=200)
        self.columnconfigure(1, weight=2, minsize=700)
        self.columnconfigure(2, weight=1, minsize=350)
        self.rowconfigure(0, weight=1)

        self.graph = graph
        self.node_clicked = None
        self.answer_graph = None
        self.tree = None
        self.path_length_entry = None
        self.path_text = None
        self.output_canvas = None
        self.modification_var = tk.BooleanVar(value=False)
        self.time_entry = None #

        self.create_widgets()
        self.draw_initial_graph()

    def create_widgets(self):
        """ """
        BG_COLOR = "#F0F0F0"
        ACCENT_1 = "#FF6B6B"
        ACCENT_2 = "#4ECDC4"
        DARK_TEXT = "#2D3436"
        LIGHT_TEXT = "#FFFFFF"
        PANEL_COLOR = "#FFEAA7"
        BTN_COLOR = "#55EFC4"

```

```

CANVAS_BG = "#DFE6E9"

self.configure(bg=BG_COLOR)

style = ttk.Style()
style.theme_use("default")
style.configure("Treeview.Heading", background=ACCENT_1,
    ↪ foreground=LIGHT_TEXT, font=('Helvetica', 10, 'bold'))
style.configure("Treeview", background="#FFFFFF",
    ↪ fieldbackground="#FFFFFF", foreground=DARK_TEXT)
style.map("Treeview", background=[('selected', ACCENT_2)])

left_frame = tk.LabelFrame(
    self,
    padx=10,
    pady=10,
    text="      ",
    bg=PANEL_COLOR,
    fg=DARK_TEXT,
    font=('Helvetica', 12, 'bold')
)
left_frame.grid(row=0, column=0, sticky="nsew")

modification_frame = tk.Frame(left_frame, bg=PANEL_COLOR)
modification_frame.pack(fill="x", pady=5)

tk.Label(
    modification_frame,
    text="      :",
    bg=PANEL_COLOR,
    fg=DARK_TEXT,
    font=('Helvetica', 10)
).pack(side="left")

modification_check = tk.Checkbutton(
    modification_frame,
    variable=self.modification_var,
    text="      ",
    bg=PANEL_COLOR,
    fg=DARK_TEXT,

```

```

        selectcolor=ACCENT_2,
        activebackground=PANEL_COLOR,
        activeforeground=DARK_TEXT,
        font=('Helvetica', 10)
    )
    modification_check.pack(side="right")

#
tk.Button(
    left_frame,
    text="",
    command=self.start_algorithm,
    bg=BTN_COLOR,
    fg=DARK_TEXT,
    activebackground=ACCENT_2,
    font=('Helvetica', 12, 'bold'),
    relief="flat"
).pack(fill="x", pady=10)

#
tk.Label(
    left_frame,
    text="",
    bg=PANEL_COLOR,
    fg=DARK_TEXT,
    font=('Helvetica', 10)
).pack(anchor="w")

self.time_entry = tk.Entry(
    left_frame,
    state="readonly",
    bg="#FFFFFF",
    fg=ACCENT_1,
    font=('Helvetica', 12, 'bold'),
    justify="center",
    readonlybackground="#FFFFFF"
)
self.time_entry.pack(fill="x", pady=2)
self.time_entry.bind("<Button-3>", self.show_context_menu)
self.time_entry.bind("<Control-c>", lambda e: self.copy_time())

```

```

if platform.system() == "Darwin":
    self.time_entry.bind("<Command-c>", lambda e: self.copy_time())

#
tk.Label(
    left_frame,
    text="          ",
    bg=PANEL_COLOR,
    fg=DARK_TEXT,
    font=('Helvetica', 10)
).pack(anchor="w")

self.path_text = tk.Text(
    left_frame,
    height=5,
    state="disabled",
    bg="#FFFFFF",
    fg=DARK_TEXT,
    font=('Courier New', 14)
)
self.path_text.pack(fill="both", pady=5, expand=True)

# self.path_text = tk.Text(
#     left_frame,
#     height=5,
#     state="disabled",
#     bg="#FFFFFF",
#     fg=DARK_TEXT,
#     font=('Courier New', 14),
#     wrap=tk.NONE,
#     selectbackground=ACCENT_2,
#     inactiveselectbackground=ACCENT_2,
#     exportselection=1,
#     highlightthickness=0,
#     borderwidth=2,
#     relief="solid"
# )
# self.path_text.pack(fill="both", pady=5, expand=True)
# self.path_text.bind("<Button-3>", self.show_text_context_menu)

```

```

# self.path_text.bind("<Control-c>", lambda e:
    ↪ self.copy_path_text())
# self.path_text.bind("<Control-Insert>", lambda e:
    ↪ self.copy_path_text())
# if platform.system() == "Darwin":
#     self.path_text.bind("<Command-c>", lambda e:
        ↪ self.copy_path_text())

#
tk.Label(
    left_frame,
    text="      ",
    bg=PANEL_COLOR,
    fg=DARK_TEXT,
    font=('Helvetica', 10)
).pack(anchor="w")

self.path_length_entry = tk.Entry(
    left_frame,
    state="readonly",
    bg="#FFFFFF",
    fg=ACCENT_1,
    font=('Helvetica', 12, 'bold'),
    justify="center"
)
self.path_length_entry.pack(fill="x", pady=2)

#
center_frame = tk.Frame(self, bg=BG_COLOR)
center_frame.grid(row=0, column=1, sticky="nsew")

input_graph_frame = tk.LabelFrame(
    center_frame,
    text="      ",
    padx=10,
    pady=10,
    bg=BG_COLOR,
    fg=DARK_TEXT,
    font=('Helvetica', 12, 'bold')
)

```

```

input_graph_frame.pack(fill="both", expand=True, side=tk.TOP)

self.canvas = tk.Canvas(
    input_graph_frame,
    bg=CANVAS_BG,
    highlightthickness=0
)
self.canvas.pack(fill="both", expand=True)

clear_button = tk.Button(
    center_frame,
    text="          ",
    command=self.clear_graph,
    bg=ACCENT_1,
    fg=DARK_TEXT,
    activebackground="#FF8787",
    font=('Helvetica', 10),
    height=2,
)
clear_button.pack(fill="x", pady=5)

output_graph_frame = tk.LabelFrame(
    center_frame,
    text="          ",
    padx=10,
    pady=10,
    bg=BG_COLOR,
    fg=DARK_TEXT,
    font=('Helvetica', 12, 'bold')
)
output_graph_frame.pack(fill="both", expand=True, side=tk.TOP)

self.output_canvas = tk.Canvas(
    output_graph_frame,
    bg=CANVAS_BG,
    highlightthickness=0
)
self.output_canvas.pack(fill="both", expand=True)

self.canvas.bind("<Button-1>", self.on_canvas_click)

```

```

#
right_frame = tk.LabelFrame(
    self,
    text="    ",
    padx=10,
    pady=10,
    bg=PANEL_COLOR,
    fg=DARK_TEXT,
    font=('Helvetica', 12, 'bold')
)
right_frame.grid(row=0, column=2, sticky="nsew")

columns = ("from", "to", "weight", "visited")
self.tree = ttk.Treeview(
    right_frame,
    columns=columns,
    show="headings",
    height=25,
)

for col in columns:
    self.tree.heading(col, text=col.capitalize())
    self.tree.column(col, width=80, anchor="center")

scrollbar = ttk.Scrollbar(
    right_frame,
    orient="vertical",
    command=self.tree.yview
)
self.tree.configure(yscrollcommand=scrollbar.set)

self.tree.pack(side="left", fill="both", expand=True)
scrollbar.pack(side="right", fill="y")

#
def copy_to_clipboard(self, text):
    self.clipboard_clear()
    self.clipboard_append(text)
    if platform.system() == "Darwin":

```

```

        self.update()

def show_context_menu(self, event):
    menu = tk.Menu(self, tearoff=0)
    menu.add_command(label="      ", command=self.copy_time)
    menu.tk.call("tk_popup", menu, event.x_root, event.y_root)

def show_text_context_menu(self, event):
    menu = tk.Menu(self, tearoff=0)
    menu.add_command(label="      ", command=self.copy_path_text)
    menu.tk.call("tk_popup", menu, event.x_root, event.y_root)

def copy_time(self):
    text = self.time_entry.get()
    self.copy_to_clipboard(text)

def copy_path_text(self):
    try:
        text = self.path_text.get("sel.first", "sel.last")
    except tk.TclError:
        text = self.path_text.get("1.0", "end")
    self.copy_to_clipboard(text)

#
def on_canvas_click(self, event):
    x, y = event.x, event.y
    node = self.get_node_at(x, y)

    if node is None:
        if self.node_clicked is None:
            self.create_node(x, y)
        else:
            print("      .")
            self.node_clicked = None
    else:
        if self.node_clicked is None:
            print(f"      {node.value}.      .")
            self.node_clicked = node
        else:
            self.create_edge(self.node_clicked, node)

```



```

        self.node_clicked = None

def get_node_at(self, x, y):
    for node in self.graph.nodes:
        if (abs(node.x - x) < 20) and (abs(node.y - y) < 20):
            return node
    return None

def create_node(self, x, y):
    node = self.graph.add_node(len(self.graph.nodes))
    node.x = x
    node.y = y
    self.canvas.create_oval(x - 15, y - 15, x + 15, y + 15, fill="red")
    self.canvas.create_text(x, y, text=str(node.value), fill="white")
    print(f"          {node.value} ({x}, {y})")

def create_edge(self, from_node, to_node):
    if from_node == to_node:
        print("          (          ).")
        return

    weight = simpledialog.askinteger("          ", f"
    ↪ {from_node.value} -> {to_node.value}")
    if weight is not None:
        self.graph.add_edge(from_node, to_node, weight)
        self.draw_edge(from_node, to_node, weight)
        self.update_edges_table()
        print(f"          {from_node.value} -> {to_node.value}
        ↪ {weight}")

def draw_edge(self, from_node, to_node, weight):
    offset = 15

    dx = to_node.x - from_node.x
    dy = to_node.y - from_node.y
    angle = math.atan2(dy, dx)

    x1, y1 = from_node.x + offset * math.cos(angle), from_node.y +
    ↪ offset * math.sin(angle)

```

```

x2, y2 = to_node.x - offset * math.cos(angle), to_node.y - offset *
    ↪ math.sin(angle)

self.canvas.create_line(x1, y1, x2, y2, arrow=tk.LAST, width=3,
    ↪ fill="black")
mid_x, mid_y = (x1 + x2) / 2, (y1 + y2) / 2
self.canvas.create_text(mid_x, mid_y, text=str(weight),
    ↪ fill="black")

def clear_graph(self):
    self.graph = Graph(directed=True)
    self.answer_graph = None
    self.canvas.delete("all")
    self.output_canvas.delete("all")
    self.update_edges_table()
    print("      .")

def start_algorithm(self):
    if self.modification_var.get():
        print("      ")
        self.graph.display()
        results = []
        start_time = time.time()

        for start_node in self.graph.nodes:
            result_graph, total_weight, result_mes, path_str =
                ↪ nearest_neighbor_method(
                    self.graph,
                    start_node=start_node
                )
            if " " in result_mes:
                results.append((result_graph, total_weight, result_mes,
                    ↪ path_str))

        end_time = time.time()
        execution_time = end_time - start_time

        if not results:
            messagebox.showerror(" ", " ")
        return

```

```

        best_result = min(results, key=lambda x: x[1])
        self.answer_graph, answer_path_len, result_mes, path_str =
            ↪ best_result
    else:
        print(" ")
        self.graph.display()
        start_time = time.time()
        self.answer_graph, answer_path_len, result_mes, path_str =
            ↪ nearest_neighbor_method(self.graph)
        end_time = time.time()
        execution_time = end_time - start_time

#
self.time_entry.configure(state="normal")
self.time_entry.delete(0, tk.END)
self.time_entry.insert(0, f"{execution_time:.4f} ")
self.time_entry.configure(state="readonly")

self.draw_graph(self.output_canvas, self.answer_graph)
self.update_edges_table()
messagebox.showinfo(" ", result_mes)

print(path_str)
self.path_text.configure(state="normal")
self.path_text.delete("1.0", tk.END)
self.path_text.insert(tk.END, path_str)
self.path_text.configure(state="disabled")

self.path_length_entry.configure(state="normal")
self.path_length_entry.delete(0, tk.END)
self.path_length_entry.insert(0, str(answer_path_len))
self.path_length_entry.configure(state="readonly")

def draw_graph(self, canvas, graph):
    canvas.delete("all")

    for edge in graph.edges:
        self.draw_edge_on_canvas(canvas, edge.from_node, edge.to_node,
            ↪ edge.weight)

```

```

    for node in graph.nodes:
        self.draw_node_on_canvas(canvas, node)

def draw_node_on_canvas(self, canvas, node):
    x, y = node.x, node.y
    canvas.create_oval(x - 15, y - 15, x + 15, y + 15, fill="blue" if
        ↪ canvas == self.output_canvas else "red")
    canvas.create_text(x, y, text=str(node.value), fill="white")

def draw_edge_on_canvas(self, canvas, from_node, to_node, weight):
    offset = 15
    dx = to_node.x - from_node.x
    dy = to_node.y - from_node.y
    angle = math.atan2(dy, dx)

    x1 = from_node.x + offset * math.cos(angle)
    y1 = from_node.y + offset * math.sin(angle)
    x2 = to_node.x - offset * math.cos(angle)
    y2 = to_node.y - offset * math.sin(angle)

    canvas.create_line(x1, y1, x2, y2, arrow=tk.LAST, width=3,
        fill="darkgray" if canvas == self.output_canvas else
        ↪ "black")
    mid_x, mid_y = (x1 + x2) / 2, (y1 + y2) / 2
    canvas.create_text(mid_x, mid_y, text=str(weight), fill="black")

def update_edges_table(self):
    for item in self.tree.get_children():
        self.tree.delete(item)

    used_edges = set()
    if self.answer_graph:
        for edge in self.answer_graph.edges:
            key = (edge.from_node.value, edge.to_node.value, edge.weight)
            used_edges.add(key)

    sorted_edges = sorted(
        self.graph.edges,
        key=lambda edge: (edge.from_node.value, edge.to_node.value)
    )

```

```

)

for edge in sorted_edges:
    from_val = edge.from_node.value
    to_val = edge.to_node.value
    weight = edge.weight
    is_used = (from_val, to_val, weight) in used_edges

    self.tree.insert("", "end", values=(
        from_val,
        to_val,
        weight,
        " " if is_used else ""
    ))

def draw_initial_graph(self):
    for node in self.graph.nodes:
        self.canvas.create_oval(
            node.x - 15, node.y - 15,
            node.x + 15, node.y + 15,
            fill="red"
        )
        self.canvas.create_text(node.x, node.y, text=str(node.value),
                                ↪ fill="white")

    for edge in self.graph.edges:
        self.draw_edge(edge.from_node, edge.to_node, edge.weight)

if __name__ == "__main__":
    graph = Graph(directed=True)
    app = LabApp(graph)
    app.mainloop()

```

9 Полезные ссылки

- Репозиторий с лабораторными работами за 4-й семестр